



35 Racing

Documentación del proceso de desarrollo

Proyecto final de Programación sobre Redes

Videojuego de carreras 2D multijugador en red local (Java 17 + libGDX)

Repositorio: <https://github.com/joakinkong/35-Racing>

Documento actualizado hasta la etapa 6 · generado el 29/09/2026 · versión 0d4bf9f

Integrantes

Joaquin Barreira

Madai Andacaba

Antonella Vivacqua

Joaquin Rodrigues

Mateo Guinsburg

Contenido

Cómo trabajamos y línea de tiempo

Etapa 0: Preparación

Etapa 1: Pre-entrega 1

Etapa 2: Esqueleto y pantallas

Etapa 3: Auto y circuito (1 jugador local)

Etapa 4: Reglas de carrera y dos autos locales

Etapa 5: Diseño del protocolo de red

Etapa 6: Conexión y lobby por TCP

Cómo fuimos trabajando: proceso de desarrollo de 35 Racing

Este documento muestra el proceso real de trabajo del proyecto, etapa por etapa: qué se pidió, qué se hizo, qué decisiones se tomaron, qué problemas aparecieron y cómo se comprobó que funcionaba. Está escrito para que se pueda seguir en orden y verificar contra el historial de Git del repositorio.

Cómo trabajamos

Uso de inteligencia artificial. El proyecto se desarrolla con Claude Code (un asistente de IA que trabaja dentro del repositorio) siguiendo una guía de prompts por etapas. La IA escribe y revisa código y documentación; el grupo decide qué se hace, aprueba los planes, prueba el juego y pide cambios. Los commits se hacen desde la cuenta de Joaquín Barreira y no llevan la línea `Co-Authored-By`, por lo que la IA no figura como colaboradora en GitHub: su uso se declara acá.

El ciclo que se repitió en cada etapa fue este:

- 1. Plan primero.** Antes de escribir código, el asistente muestra un plan (clases a crear o modificar y decisiones a tomar) y se espera la confirmación del grupo. Es una regla del archivo `CLAUDE.md`.
- 2. Rama de trabajo.** Cada etapa se hace en una rama `feature/...` que sale de `develop`.
- 3. Implementación.** Código en español, con constantes en `Config` y `ParametrosAuto` (sin números mágicos) y el modelo del juego separado de lo gráfico.
- 4. Verificación.** Además de compilar con Gradle, se escribieron pruebas descartables que abren el juego con la ventana oculta, simulan autos con entradas guionadas y sacan capturas de pantalla. Los resultados numéricos de cada prueba están en los documentos de cada etapa.
- 5. Corrección.** Los errores que encontró la prueba o el grupo al jugar se corrigieron en commits propios, con prefijo `Fix:`.
- 6. Integración.** Pull request de la rama de trabajo a `develop`, y de `develop` a `main` al cerrar un hito. El `CHANGELOG` se actualiza en cada etapa.

Los commits usan los prefijos `Feat:`, `Fix:`, `Docs:` y `Refactor:`, como pide la guía.

Línea de tiempo

Las etapas 0 a 4 se hicieron el 28/09/2026, entre las 21:16 y las 23:45 (hora local, según los commits). Esta documentación del proceso se armó a continuación, y las etapas 5 y 6 el 29/09/2026.

Etapa	Qué quedó funcionando	Pull request
0. Preparación	Proyecto LibGDX (core + lwjgl3) con Java 17, repositorio público en GitHub con ramas <code>main</code> y <code>develop</code>	(commits directos)
1. Pre-entrega 1	README, CHANGELOG, <code>.gitignore</code> y la propuesta pasada a Markdown para la Wiki	#1

Etapa	Qué quedó funcionando	Pull request
2. Esqueleto y pantallas	Menú, unirse, lobby, carrera y resultados navegables, con estética de Fórmula 1	#2
3. Auto y circuito	Un auto manejable en un circuito, con física arcade, cámara y HUD	#3
4. Reglas de carrera	Carrera completa para 2 jugadores en el mismo teclado: cuenta regresiva, vueltas, posiciones, choques y resultados	#4 y #5
Documentación del proceso	Esta documentación por etapas y su versión en PDF	#6
5. Protocolo de red	Especificación completa de la comunicación en red en docs/PROTOCOLO.md (diseño, todavía sin código)	#7
6. Conexión y lobby por TCP	Crear una partida, unirse por IP, lobby en vivo con elección de auto y "listo", e inicio de la carrera (cada uno todavía corre la suya)	Pendiente

Estado del repositorio al cerrar la etapa 6: `develop` tiene el protocolo (etapa 5) y `main` todo hasta la etapa 4 y esta documentación; la etapa 6 está en la rama `feature/lobby-tcp`.

Lo que sigue

- **Antes de seguir:** que los cinco integrantes lean [docs/PROTOCOLO.md](#), porque es lo que tienen que poder explicar.
- **Ahora:** probar la etapa 6 con varias instancias y, cuanto antes, en las computadoras del colegio: si la red bloquea algo, mejor enterarse ahora.
- **Etapas 7 y 8:** carrera sincronizada por UDP, desconexiones y pruebas en varias computadoras.
- **Pendiente de arte:** el circuito definitivo en Tiled y los sprites. Hoy el circuito y los autos son provisionales (ver [etapa 3](#)).

Participación del equipo

Hasta la etapa 6 todos los commits salieron de la cuenta de Joaquín Barreira. El resto del grupo tiene invitación como colaborador del repositorio, y las tareas de arte (circuito y sprites) están asignadas al resto del equipo según la guía. Cuando haya commits de los demás integrantes, se van a reflejar en el historial y en esta documentación.

Etapa 0: Preparación

Objetivo

Dejar listo el entorno y el proyecto base, según la guía: herramientas instaladas, proyecto generado con gdx-liftoff, importado en Eclipse, documentos de la propuesta dentro del repositorio y repositorio público en GitHub.

Qué se hizo

Herramientas. Se verificó lo que ya estaba instalado: JDK 21 (cumple el mínimo de Java 17), Git 2.53 con el nombre y el mail configurados, y la CLI de GitHub (`gh`), que después se usó para crear ramas y gestionar colaboradores.

Proyecto con gdx-liftoff (versión 1.14.2.3), con estas opciones:

Opción	Valor
Nombre del proyecto	35 Racing
Paquete	<code>ar.edu.et35.racing</code>
Clase principal	<code>Main</code>
Plataformas	Core y Desktop (LWJGL3), nada más
Extensiones	Freetype únicamente (sin Box2D ni Ashley)
Plantilla	Classic

El proyecto se importó en Eclipse como *Existing Gradle Project* y `Lwjgl3Launcher` abrió la ventana con el logo de libGDX. Antes de importarlo se comprobó que compilara con Gradle (`gradlew lwjgl3:classes` , sin errores).

Documentos del repositorio.

- `docs/propuesta/propuesta.pdf` : la propuesta aprobada (versión 2), con un nombre sin espacios para poder referenciarla.
- `docs/propuesta/img/` : las tres figuras de la propuesta.
- `CLAUDE.md` : el contexto fijo del proyecto para el asistente (stack, arquitectura, convenciones y forma de trabajo), con el nombre y el paquete ya completados.

Repositorio. [joakinkong/35-Racing](https://github.com/joakinkong/35-Racing), público (necesario para tener Wiki) y con la Wiki activada. Ramas `main` y `develop` . Se invitó como colaboradores a `jasinski1988` (profesor), `JoacoR2007` , `mateoguinsburg35` y `antonellavivacqua` .

Decisiones

- **Paquete** `ar.edu.et35.racing` : sigue la convención de dominio invertido e identifica a la escuela.
- **Repositorio** `35-Racing` : GitHub no acepta espacios en el nombre. Cuando se fue a crear ya existía uno vacío con ese nombre en la misma cuenta, y se usó ese en lugar de crear otro.
- **Java 17 en Gradle:** `gdx-liftoff` genera el proyecto compilando para Java 8, pero las pautas piden Java 17. Se cambió `sourceCompatibility` , `targetCompatibility` y `release` a 17 en `build.gradle` y `lwjgl3/build.gradle` , y se volvió a compilar. Con Java 8 no se habrían podido usar funciones del lenguaje como los `record` , que el proyecto usa más adelante.

Problemas encontrados

- `gh` no tenía sesión iniciada: se resolvió con `gh auth login` .
- El repositorio ya existía vacío: se verificó que estaba vacío, era público y tenía la Wiki activa antes de usarlo.

Commits

- `9b1bca7` : Configuración inicial del proyecto con LibGDX Liftoff.
- `ecaae86` : Fix: Compilar con Java 17 según las pautas de la materia.

Etapa 1: Pre-entrega 1

Objetivo

Completar la documentación mínima de la primera pre-entrega: propuesta en la Wiki, `.gitignore`, CHANGELOG y README.

Qué se hizo

- **Propuesta en Markdown para la Wiki.** Se extrajo el texto del PDF de la propuesta y se pasó a `docs/wiki/Propuesta-del-Proyecto.md`, respetando el texto y la numeración de secciones. La portada quedó resumida en un encabezado con el título, la fecha y los integrantes. Se dejaron afuera la tabla de contenidos y la sección de aprobación, y se insertaron las tres figuras con su epígrafe. Después la página se cargó a mano en la Wiki de GitHub.
- **`.gitignore`.** Se revisó el que generó liftoff contra la lista pedida: ya cubría todo (Gradle, `bin/`, `.project`, `.classpath`, `.settings/`, `.idea/`, `Thumbs.db`, etc.) salvo `.claude/settings.local.json`, que se agregó.
- **CHANGELOG.** Se creó `CHANGELOG.md` en español con formato [Keep a Changelog](#) y una entrada `[0.1.0]` para el proyecto generado, el repositorio, el README, el CHANGELOG y la Wiki.
- **README.** Se reemplazó el README genérico de liftoff por uno propio: nombre, integrantes, descripción breve basada en la propuesta, tecnologías, enlace a la Wiki, cómo compilar y ejecutar (Eclipse y consola) y estado actual.

Decisiones

- **Fecha del CHANGELOG en formato ISO** (`2026-09-28`), que es el que usa Keep a Changelog.
- **README solo con lo que ya existe:** describe el estado real ("configuración inicial y estructura del proyecto") y no promete funciones que todavía no están.

Problemas encontrados

- **Lectura del PDF.** La herramienta de lectura de PDF del asistente necesita `poppler`, que no está instalado. Se resolvió extrayendo el texto con la biblioteca `pypdf` de Python, que además hizo falta configurar con codificación UTF-8 para que no fallaran los caracteres especiales.
- **Link roto en el CHANGELOG.** El CHANGELOG enlazaba al tag `v0.1.0`, que nunca se creó, así que daba error 404. Se quitó el enlace en lugar de crear un tag que nadie había pedido ([6436374](#)).

Checklist de la pre-entrega 1 (estado al cerrar la etapa)

Ítem	Estado
El proyecto compila y corre en escritorio	Cumplido

Ítem	Estado
<code>.gitignore</code> adecuado	Cumplido
README con nombre, integrantes, descripción, tecnologías, Wiki e instrucciones	Cumplido
CHANGELOG con la entrada inicial	Cumplido
Wiki activada, con la propuesta	Cumplido
Todos los integrantes y los 3 profesores como colaboradores	Incompleto: había 4 invitaciones enviadas y faltaban Madai y dos de los profesores
Commits de varios integrantes	Incompleto: todos los commits eran de una sola cuenta

Commits y pull request

- `85c0daf` : Docs: README, CHANGELOG y `.gitignore` para pre-entrega 1.
- `a023fda` : Docs: Propuesta del proyecto en Markdown para la Wiki.
- `6436374` : Docs: Quitar link roto al tag v0.1.0 del CHANGELOG.
- Pull request `#1`: `develop` a `main`.

Etaapa 2: Esqueleto y pantallas

Objetivo

Amar el esqueleto del juego, todavía sin física ni red: cinco pantallas navegables con botones (menú principal, unirse, lobby, carrera y resultados), con ESC para volver, una interfaz hecha con Scene2D y una resolución virtual de 640×360 con estilo pixel art. Además, el grupo pidió que todo el juego tenga una **estética de Fórmula 1**.

El plan (se mostró y se confirmó antes de programar)

Clase	Qué hace
<code>util/Config</code>	Constantes globales (<code>MAX_JUGADORES</code> = 5 , <code>VUELTAS</code> = 3 , <code>TICKS_POR_SEGUNDO</code> = 60 , puertos 7777 y 7778, resolución virtual, ventana inicial)
<code>util/Paleta</code>	Colores del estilo visual
<code>util/Recursos</code>	Punto único que crea y libera el <code>AssetManager</code> , el <code>SpriteBatch</code> , las fuentes y el <code>Skin</code>
<code>pantallas/PantallaBase</code>	Clase base con la escena, el <code>FitViewport</code> de 640×360, el manejo de ESC y el marco visual
<code>MenuPrincipal</code> , <code>UnirsePartida</code> , <code>Lobby</code> , <code>Carrera</code> , <code>Resultados</code>	Las cinco pantallas
<code>Main</code>	Pasa a extender <code>Game</code> y decide qué pantalla se muestra

También se crearon los paquetes `juego` , `vista` , `red` , `pantallas` y `util` , cada uno con un `package-info.java` de una línea que dice qué va ahí.

Qué se hizo

- **Interfaz sin archivos externos.** El `Skin` se arma por código: la fuente por defecto de libGDX y rectángulos de color como fondos. Para que el pixel art no se vea borroso al escalar se usa filtro *Nearest*.
- **Resolución virtual 640×360** con `FitViewport` . La ventana abre en 1280×720, es redimensionable y no baja de 640×360; al agrandarla, el contenido escala con barras a los costados en lugar de deformarse.
- **Estética de Fórmula 1:** fondo color carbono (`#15151E`), rojo de largada (`#E10600`) y blanco; una bandera de cuadros arriba y abajo de cada pantalla; cinco luces rojas de largada bajo el título del menú; una grilla de largada P1 a P5 en el lobby; y pianos rojo y blanco en el fondo de la carrera.
- **Navegación completa** entre las cinco pantallas, con ESC para volver al menú.

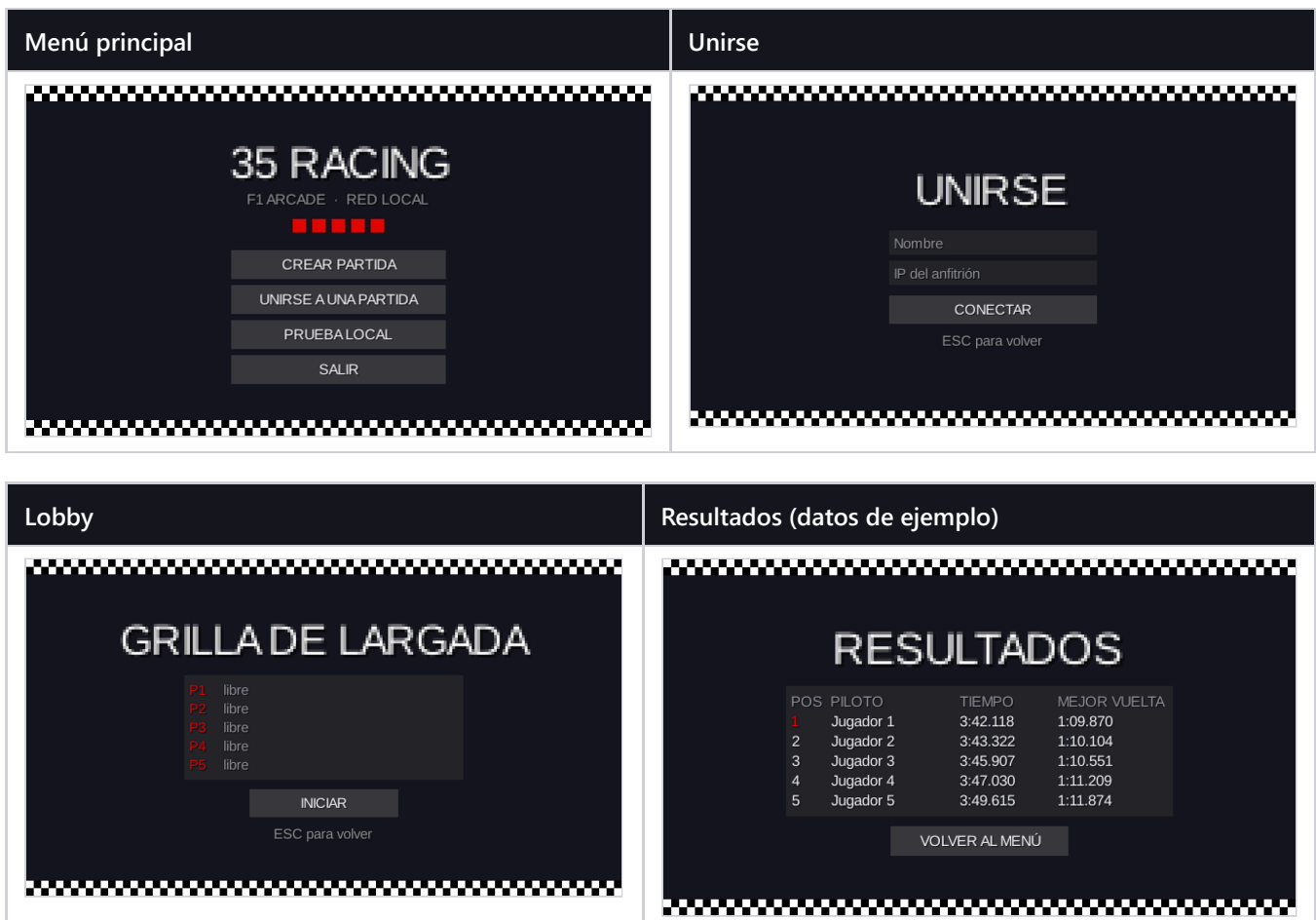
Decisiones

- `PantallaBase` **compartida**. Las cinco pantallas repiten la misma estructura (escena, ESC, marco). Se resolvió una sola vez en una clase base.
- **Un solo** `SpriteBatch`. Las escenas comparten el de `Recursos` en lugar de crear uno por pantalla, y así se libera en un único lugar.
- **Cambio de pantalla diferido**. `Main.irA(...)` cambia de pantalla con `Gdx.app.postRunnable`. Se llama desde un botón de la pantalla vieja, y destruirla en medio de su propio evento puede fallar.
- **Sin acentos que no existan en la fuente**. Antes de escribir los textos se verificó que la fuente por defecto trae ó, é, ñ, ¡ y Ó, pero no las flechas; por eso no se usan flechas en la interfaz.
- **Botón "Finalizar carrera" provisorio** para poder llegar a Resultados sin lógica de carrera. Se quitó en la etapa 4.

Verificación

- Compila con Gradle sin errores.
- El juego arrancó y siguió corriendo 25 segundos sin excepciones (se detuvo desde afuera).
- Se sacaron capturas de las pantallas con una ventana oculta.

Capturas



Commits y pull request

- `c007aca` : Feat: Config, Paleta y Recursos con Skin por código y estilo F1.
- `76eebb3` : Feat: Esqueleto de pantallas con navegación y ventana 1280x720.
- Pull request [#2](#).

Una lección de flujo de trabajo

El pull request #2 se hizo directamente contra `main`, en lugar de contra `develop`, y así se saltó la rama de integración. No se rompió nada, pero después se corrigió la práctica: los pull requests de cada etapa van a `develop`, y `develop` pasa a `main` al cerrar un hito (ver etapas siguientes).

Eta3a 3: Auto y circuito (1 jugador local)

Objetivo

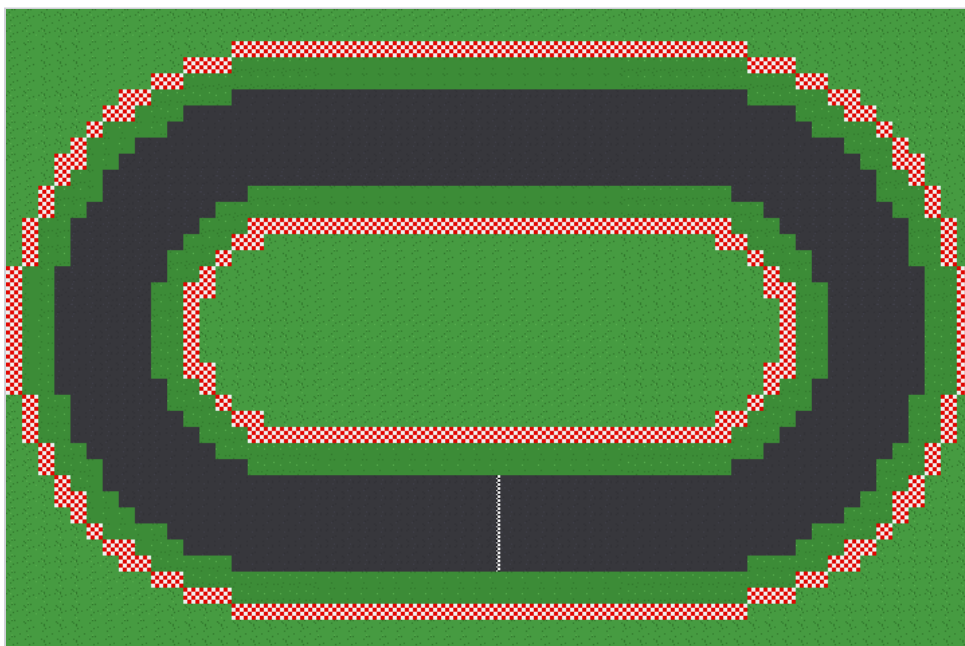
Un auto manejable en un circuito, con un solo jugador local y sin red: física arcade propia (sin Box2D), circuito cargado desde un mapa de Tiled, cámara que sigue al auto y simulación a paso fijo desacoplada del dibujo.

El circuito: un requisito que faltaba

La guía indica que la persona de arte arma a mano `assets/circuitos/circuito1.tmx` en Tiled, con tiles de 32×32 y propiedades personalizadas. Ese archivo todavía no existía, y sin él no se podía probar nada. Se propuso y se aprobó generar un **circuito provisorio** con un script, con el mismo formato exacto que pide la guía, para poder seguir sin bloquearse:

- Un óvalo de 60×40 tiles con pista de 6 tiles de ancho, pasto de escape de 2 tiles a cada lado y muros con pianos rojo y blanco.
- Un tileset de 5 tiles en pixel art (pasto, pista, muro, línea de meta y una variante de pasto) generado por código.
- Capa de tiles `suelo` con la propiedad `superficie` (`pista`, `pasto` o `muro`) en cada tile, capa de objetos `checkpoints` (10 rectángulos con `orden`, donde el 0 es la meta), capa `largada` (5 lugares con `posicion`) y la propiedad `anguloLargada` en el mapa.

Cuando arte tenga el circuito definitivo, lo reemplaza sin tocar código mientras respete esas capas y propiedades.



Vista general del circuito provisorio

Qué se hizo

Modelo (juego), sin ninguna clase gráfica porque después lo usa el servidor:

Clase	Qué hace
Circuito	Datos puros: grilla de superficies, checkpoints ordenados, lugares y ángulo de largada; detecta si un círculo choca con un muro
CargadorCircuito	Convierte el TiledMap en un Circuito y valida que el mapa tenga todas las capas y propiedades
Superficie	PISTA , PASTO o MURO
EntradaAuto	Lo que hace el jugador: acelerar, frenar y girar (-1, 0 o 1)
ParametrosAuto	Todos los valores de la física en un solo lugar
Auto	Posición, ángulo y velocidad, con la física arcade

Física del auto. La velocidad se separa en una componente hacia adelante y otra lateral respecto de adonde mira el auto, y la lateral se va perdiendo de a poco: de ahí sale el derrape leve. Tiene aceleración, freno, reversa lenta, velocidad máxima y rozamiento; el giro depende de la velocidad (parado no gira, en reversa gira al revés); en el pasto baja la velocidad máxima y sube el rozamiento; y contra un muro rebota y pierde velocidad sin atravesarlo. La colisión se resuelve un eje por vez, con el auto como círculo.

Vista (vista). VistaCircuito dibuja el mapa, VistaAuto dibuja el auto como un monoplaa armado con rectángulos (hasta que haya sprites) y CamaraSeguimiento sigue al auto con un leve adelanto hacia donde va.

Pantalla Carrera . Simulación a paso fijo de 1/60 s con un acumulador, y dibujo que interpola entre el último tick y el siguiente para que el movimiento sea fluido aunque la frecuencia de cuadros varíe. Se maneja con flechas o WASD.

Decisiones

- **Física propia y sin Box2D:** lo exigen las pautas.
- **Modelo separado de lo gráfico:** Circuito , Auto y EntradaAuto no usan nada de dibujo ni de teclado, para que el servidor pueda simular la carrera.
- **Colisión por eje:** si el auto chocaría al moverse en un eje, ese eje no se mueve y su velocidad rebota con pérdida. Así no atraviesa muros ni se traba.
- **Tope de tiempo por cuadro (0,25 s):** un tirón, por ejemplo al mover la ventana, no dispara cientos de ticks de golpe.

Problemas encontrados y cómo se resolvieron

1. **En el pasto con el acelerador apretado, el auto se frenaba por completo.** Se detectó al revisar el código antes de correrlo: el rozamiento del pasto (390 px/s²) le ganaba a la aceleración (220 px/s²). Se

cambió para que en el pasto con el acelerador la velocidad la limite el máximo permitido y el auto avance despacio.

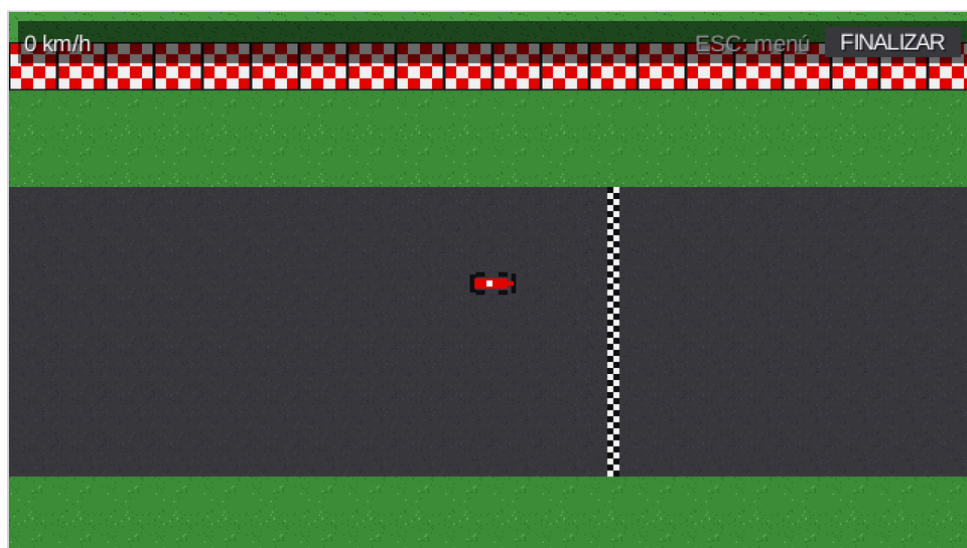
2. **Marcha atrás imposible en el pasto.** Lo encontró la prueba automática: un auto pegado a un muro, en la zona de pasto, no podía salir marcha atrás, porque el rozamiento por tick (5 px/s) era mayor que la aceleración en reversa por tick (2 px/s). Se corrigió para que el rozamiento solo actúe cuando no se toca ningún pedal.

Verificación

Se escribió una prueba descartable (no forma parte del repositorio) que abre el juego con la ventana oculta, carga el circuito real y simula autos con entradas guionadas. Resultados:

Prueba	Resultado
El <code>.tmx</code> carga	1920×1280 px, 10 checkpoints, 5 lugares de largada, todos sobre pista y sin tocar muros
Acelerar 3 s en recta	Llega a la velocidad máxima de 360 px/s
Parado con el giro apretado	El ángulo no cambia
Frenar parado	Retrocede, con tope de 90 px/s
Pasto acelerando	162 px/s, que es el 45 % del máximo, como se configuró
Chocar de frente 8 s contra un muro	0 veces dentro del muro; después sale marcha atrás (antes del arreglo, se quedaba trabado)
Piloto automático de 2 vueltas	Completa las 2 vueltas (11,7 s y 10,7 s) sin tocar un muro

También se sacó una captura del auto en la largada.



Auto en la línea de largada

Cómo ajustar la sensación de manejo

Solo se toca `ParametrosAuto` : `ACELERACION` y `VELOCIDAD_MAX` (arranque), `GIRO_MAX` y `VELOCIDAD_GIRO_COMPLETO` (giro), `AGARRE` (más alto, menos derrape), `ROZAMIENTO` (cuánto tarda en frenarse solo), `FACTOR_VELOCIDAD_PASTO` y `ROZAMIENTO_PASTO` (pasto), `REBOTE` y `PERDIDA_POR_ROCE` (muros).

Commits y pull request

- [9df7882](#) : Feat: Circuito provisorio circuito1.tmx con tileset en pixel art.
- [5fb156e](#) : Feat: Modelo del auto y del circuito con física arcade.
- [80d0f95](#) : Feat: Carrera con un auto manejable, cámara y HUD.
- Pull request [#3](#), esta vez sí hacia `develop` .

Etapa 4: Reglas de carrera y dos autos locales

Objetivo

Sumar las reglas de una carrera completa, todavía sin red: cuenta regresiva, vueltas validadas por checkpoints, posiciones, choques entre autos, fin de carrera y resultados. Es el paso de la propuesta en que "se incorporará el segundo vehículo controlado localmente para probar la lógica de colisiones y vueltas".

El plan (se mostró y se confirmó antes de programar)

Además de las reglas, el plan proponía tres decisiones que el grupo confirmó: renombrar la pantalla `Carrera` a `PantallaCarrera` (para no chocar con el modelo `juego.Carrera`), que "Prueba local" sea de 2 jugadores en el mismo teclado (J1 con WASD y J2 con flechas) y usar los nombres provisionales "Jugador 1" y "Jugador 2".

Qué se hizo

Modelo (`juego`), sin clases gráficas. `Carrera` recibe las entradas de cada auto por identificador (`Map<Integer, EntradaAuto>`) y avanza un paso de simulación. Es la clase que más adelante va a correr en el servidor, así que no lee el teclado. Sus reglas:

- **Estados:** `CUENTA_REGRESIVA` , `EN_CURSO` y `TERMINADA` . La cuenta dura 3 segundos y hasta el "¡YA!" las entradas se ignoran y los autos no se mueven.
- **Vueltas:** el auto sale detrás de la meta y tiene que pasar por los checkpoints en orden y cruzar la meta. Se mide el tiempo de cada vuelta, la mejor vuelta y el tiempo total.
- **Posiciones:** primero los que terminaron, por tiempo; después, por vueltas completadas, por checkpoint alcanzado y por distancia al próximo checkpoint.
- **Fin:** cuando el primero completa las 3 vueltas, el resto tiene 30 segundos para terminar. Pasado ese tiempo, o si todos terminaron antes, la carrera pasa a `TERMINADA` .
- **Choques entre autos:** dos círculos de igual masa que se separan por el solapamiento (si un muro le impide moverse a uno, el otro absorbe lo que falta) y se reparten la velocidad con un rebote parcial.

Clases nuevas: `Carrera` , `EstadoCarrera` , `Participante` y `ResultadoJugador` .

Pantalla y vista. `PantallaCarrera` maneja 1 o 2 jugadores locales, con un panel de HUD por jugador (vuelta, posición, tiempo actual y mejor vuelta, con una franja del color de su auto), un cartel grande para la cuenta regresiva y el "¡YA!", y el pase automático a Resultados 2,5 segundos después de terminar. `Resultados` muestra datos reales. Se agregaron `Tiempo` (formato `m:ss.mmm`) y una paleta con un color por auto.

Decisiones

- `Carrera` **pensada para el servidor:** sin teclado, sin dibujo y con las entradas por identificador, porque va a ser lo que simule el servidor autoritativo cuando llegue la red.

- **Un auto que terminó deja de recibir entradas** y se va frenando solo.
- **Los que no terminan** quedan ordenados por avance y su tiempo total se muestra con guiones.

Problemas encontrados y cómo se resolvieron

La verificación y el uso real encontraron varios problemas. Estos son los más importantes, en el orden en que aparecieron:

1. **Manejar en contramano avanzaba checkpoints** (encontrado revisando la regla). Con la condición "cruzar el checkpoint hacia adelante", un auto que recorre la pista al revés igualmente sumaba un checkpoint por vuelta. Se agregó una segunda condición: además de ir hacia adelante, el auto tiene que avanzar en el sentido de la pista en ese checkpoint (calculado desde la geometría del circuito).
2. **La primera prueba de contramano no probaba nada.** Reportó 0 checkpoints cruzados, porque el piloto automático de prueba no podía dar la vuelta estando parado (un auto parado no gira). Se corrigió la prueba, y con la versión correcta cruzó más de 80 checkpoints al revés sin sumar una sola vuelta.
3. **Los checkpoints solo cubrían la pista** (observación del grupo al jugar). Un auto que se salía al pasto de escape se los salteaba y no sumaba vueltas, aunque ir por el pasto ya es una desventaja porque el auto va más lento. Se regeneró el circuito con checkpoints de muro a muro. Prueba: un piloto que fue el 72 % del tiempo por el pasto completó 2 vueltas en 46,6 s, contra unos 21 s por la pista. Cuentan, pero siguen siendo más lentas.
4. **La cámara perdía de vista al segundo jugador** (observación del grupo al jugar). Con una cámara compartida, cuando los autos se separaban mucho la cámara seguía solo al que iba primero y el otro quedaba fuera de pantalla. Se probaron dos soluciones, y la segunda la pidió el grupo:
 - Primero, pantalla dividida solo cuando los autos no entraban juntos.
 - Después, por pedido del grupo, algo más simple: cada jugador tiene su propia cámara con zoom fijo de 2x y la pantalla queda siempre dividida, sin unirse ni separarse. Se eliminó bastante código: el commit quita 134 líneas y suma 38.
5. **Una captura rara que no era un error del juego.** Al probar la pantalla dividida, un auto aparecía descentrado. La causa era la propia prueba, que movía el auto de lugar durante la cuenta regresiva, cuando el auto no se actualiza; se repitió la prueba moviéndolo con la carrera en curso y quedó centrado. Sirvió para separar un artefacto de la prueba de un error real.
6. **El texto de ayuda del HUD** se perdía sobre los pianos del mapa: se le agregó un fondo, como al resto del HUD.

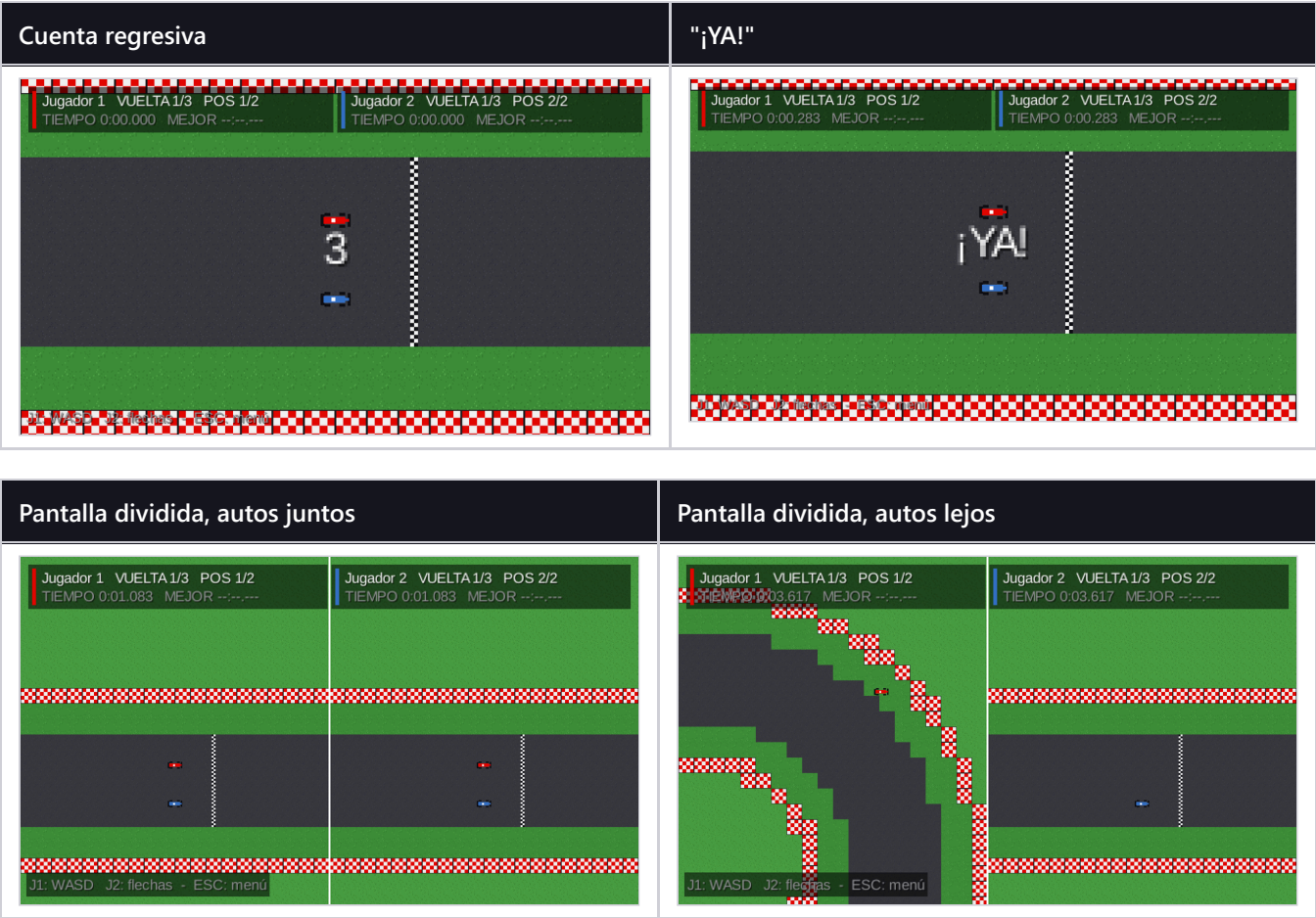
Verificación

Con la misma técnica de la etapa 3 (prueba descartable con ventana oculta), sobre el circuito real:

Prueba	Resultado
Cuenta regresiva con el acelerador apretado	Dura 3,02 s y el auto se movió 0 px
Sumar un auto con la carrera empezada	Se rechaza
Dos pilotos automáticos, 3 vueltas	Terminan en 32,866 s y 34,100 s, en el orden correcto

Prueba	Resultado
Contramano durante 90 s	83 checkpoints cruzados al revés, 0 vueltas
Choque por atrás	Distancia mínima entre autos de 20 (igual a 2 radios: no se solapan ni se traban)
Cierre de la carrera	30,02 s después de que termina el primero (se pidieron 30)
El que no terminó	Aparece en resultados sin tiempo total
Vueltas por el pasto	2 vueltas completas, 46,6 s

Capturas





Resultados con datos de la prueba con dos pilotos automáticos

Los nombres "Rapido" y "Lento" de la última captura son de la prueba con pilotos automáticos; en el juego los autos se llaman "Jugador 1" y "Jugador 2".

Commits y pull requests

- [a547b09](#) : Feat: Reglas de carrera en el modelo (cuenta, vueltas, posiciones y choques).
- [3565910](#) : Feat: Prueba local a 2 jugadores con HUD, cámara y resultados reales.
- [d05a5b0](#) : Fix: Checkpoints de muro a muro para que cuenten también por el pasto.
- [71deaca](#) : Fix: Pantalla dividida en la prueba local cuando los autos no entran juntos.
- [23a261b](#) : Fix: Zoom fijo en 2x en pantalla dividida para que el corte no cambie la escala.
- [4f385f1](#) : Refactor: Una cámara por jugador con zoom fijo 2x, sin unir ni separar pantallas.
- [c0b9567](#) : Docs: Línea en blanco antes de la entrada 0.1.0 del CHANGELOG.
- Pull request [#4](#): rama de trabajo a `develop`.
- Pull request [#5](#): `develop` a `main`, con todo lo de las etapas 3 y 4.

Etapa 5: Diseño del protocolo de red

Objetivo

Diseñar, sin escribir código, cómo se comunican las computadoras durante una partida. El resultado es [docs/PROTOCOLO.md](#): la especificación que se va a implementar en las etapas 6 a 8 y lo que el grupo tiene que poder explicar. La guía pedía cubrir la arquitectura, la máquina de estados de la partida, los mensajes TCP y UDP con su formato exacto, las frecuencias con la cuenta del ancho de banda, la interpolación en el cliente, los hilos, las fallas, la justificación de TCP y UDP y un diagrama de secuencia de una partida completa.

Para esta etapa se usó el modelo Claude Opus 5.5, como recomienda la guía para los pasos de diseño.

El plan (se mostró y se confirmó antes de escribir)

Antes de redactar el documento se presentaron las decisiones principales, y el grupo confirmó estas tres, que eran las que tenían alternativas:

Decisión	Lo elegido	Alternativa que se descartó
Frecuencia de los estados y retraso de dibujo	30 estados por segundo y 100 ms de retraso de interpolación	60 por segundo y 50 ms (menos demora, el doble de tráfico)
Predicción del auto propio	No, en la primera versión	Predecir desde el principio (más complejo)
Detección de caídas	PING cada 2 s y conexión perdida a los 6 s sin datos	(sin alternativa planteada)

Qué se decidió

- **Servidor autoritativo dentro del proceso del host**, y el host se conecta a su propio servidor por `127.0.0.1` como un cliente más, así hay un solo camino de código.
- **Un solo hilo toca el estado de la partida** (el de simulación). Los demás hilos le pasan datos por una cola de eventos (lo que llega por TCP y no se puede perder) o por un valor "última entrada" que se sobrescribe (lo que llega por UDP). Así el modelo no necesita sincronización.
- **TCP en texto** (una línea por mensaje, campos separados por `;`) para el lobby, la largada, los resultados y las desconexiones, con una tabla de 8 códigos de error.
- **UDP en binario** con dos paquetes: `ENTRADA` (11 bytes, cliente al servidor, 60 por segundo) y `ESTADO` (132 bytes con los 5 autos, servidor a todos, 30 por segundo). Cada uno lleva un número de secuencia y se descarta todo lo que llega con un número menor o igual al último.
- **El servidor aprende la dirección UDP de cada cliente** con un token que entrega por TCP en la bienvenida: guarda la dirección de origen de la primera `ENTRADA` válida.

Números que se verificaron

Los tamaños de los paquetes se comprobaron armándolos campo por campo con un script (la suma de los tamaños de cada campo), y las cuentas de ancho de banda y demora, calculándolas en el mismo script en lugar de estimarlas a mano:

Qué	Valor
ENTRADA	11 bytes (39 con las cabeceras IP y UDP)
ESTADO con 5 autos	132 bytes (160 con cabeceras); entra en un paquete sin fragmentarse (el máximo es 1.472)
Tráfico en la placa de red del host con 5 jugadores	28.560 bytes/s, unos 0,23 Mbit/s (el 0,23 % de una red de 100 Mbit/s)
Demora desde la tecla hasta ver el auto propio	unos 135 ms típica, 171 ms en el peor caso
Lo mismo con 60 estados por segundo y 50 ms	unos 77 ms típica, 104 ms en el peor caso

Problemas encontrados y cómo se resolvieron

- Los números del plan no coincidían con el diseño final.** En el plan se habían estimado 10 bytes para `ENTRADA` y 141 para `ESTADO`. Al armar los formatos campo por campo quedaron en 11 y 132 bytes (se agregó un byte de tipo a la entrada y se sacó la velocidad del estado, que el cliente puede estimar). El documento usa los números verificados y se le informó al grupo la diferencia. El cambio en el ancho de banda es mínimo.
- La demora de 30 Hz con 100 ms resultó mayor de lo que parecía.** El cálculo paso a paso mostró que casi toda la demora es el retraso de interpolación y no la red (en una LAN un paquete tarda de 1 a 5 ms en ir y volver). Se mantuvo lo confirmado, pero el documento deja el plan para la etapa 7: probar, y si la dirección se siente lenta, pasar a 60 Hz y 50 ms cambiando dos constantes; la predicción queda como último recurso.
- El servidor no puede cargar el circuito.** Al revisar el código actual contra el diseño, se vio que cargar el mapa de Tiled crea texturas, algo que en LibGDX solo se puede hacer en el hilo de render, no en un hilo del servidor. Se resolvió en el diseño: el host carga el `Circuito` y se lo pasa al servidor al crearlo.
- Decimales y configuración regional.** En una PC configurada en español, `String.format` escribe los decimales con coma, y el otro lado no los puede leer como número. Se decidió que todos los tiempos de los mensajes TCP vayan en milisegundos enteros.
- Diagramas.** Los tres diagramas Mermaid del documento (arquitectura, estados y secuencia) se renderizaron con la biblioteca oficial de Mermaid para confirmar que no tienen errores de sintaxis antes de subirlos. GitHub los muestra como gráficos.

Cambios al modelo que quedan para la etapa 7

El documento termina con una lista de 7 cosas del código actual que complican la red. Entre ellas, que la pantalla de carrera simula y dibuja a la vez (en red el cliente solo dibuja), que `VistaAuto` y la cámara necesitan un `Auto` con física, que hace falta marcar un jugador como desconectado y que el color del auto sale del id del jugador y no del auto elegido en el lobby.

Próximo paso

La guía indica que los cinco integrantes lean <docs/PROTOCOLO.md> antes de seguir con la etapa 6 (conexión y lobby por TCP), porque es lo que tienen que poder explicar.

Commits

- Documentación de la etapa en la rama `docs/protocolo-red` : el protocolo, esta página y la actualización del PDF.

Etapa 6: Conexión y lobby por TCP

Objetivo

Implementar la primera parte del protocolo de red ([docs/PROTOCOLO.md](#), secciones 1, 2, 3, 7 y 8): que una PC cree una partida, que otras se unan por IP, y que todos vean en vivo el lobby, elijan auto, marquen "listo" y el host inicie. Todavía sin UDP y sin carrera sincronizada: al iniciar, cada uno corre su propia carrera local.

El plan (se mostró y se confirmó antes de programar)

El grupo confirmó tres decisiones que el plan proponía:

1. **"Crear partida" pide el nombre.** La guía no lo decía, pero el host necesita uno para aparecer en el lobby.
2. **INICIAR exige al menos 2 jugadores**, como dice el protocolo (`Config.MIN_JUGADORES`). Para probar solo hace falta abrir 2 instancias.
3. **Al iniciar, cada cliente corre su propia carrera local** con su nombre y su color, hasta que en la etapa siguiente se sincronice.

Qué se hizo

Paquete `red` (Java puro, sin LibGDX):

Clase	Qué hace
<code>Protocolo</code>	Nombres de mensajes y de errores como constantes; arma y lee las líneas (<code>;</code> , UTF-8); valida el nombre (solo letras, números y espacios, de 1 a 12 caracteres)
<code>ConexionTcp</code>	Un <code>Socket</code> con mensajes por línea: escritura sincronizada con <code>flush</code> , <code>setTcpNoDelay(true)</code> , timeout de lectura de 6 s
<code>servidor/ServidorPartida</code>	<code>ServerSocket</code> en el puerto 7777 con tres tipos de hilos: uno que acepta, uno lector por cliente y uno de lógica
<code>cliente/ClientePartida</code>	Se conecta en segundo plano (timeout de 3 s), lee en otro hilo y deja los mensajes en una cola thread-safe; manda un <code>PING</code> cada 2 s
<code>EstadoLobby</code>	La foto del lobby (quién es el host y, por jugador, id, nombre, auto y si está listo), que sabe armarse y leerse como mensaje <code>LOBBY</code>
<code>SesionRed</code>	Lo que una PC tiene abierto: su cliente y, si es el host, también el servidor
<code>DireccionesLocales</code>	Las IPv4 de la red local de esta PC (sin las de máquinas virtuales), para mostrarlas en el lobby del host
<code>RegistroRed</code>	Log en consola de cada mensaje enviado y recibido, controlado por <code>Config.DEBUG_RED</code>

Lo que implementa el servidor: `UNIRSE` con las cinco validaciones de error (versión, carrera empezada, partida llena, nombre inválido y nombre repetido), `ELEGIR_AUTO` sin repetir, `LISTO`, `INICIAR` (solo el host y con todos listos), `SALIR`, `PING / PONG`, las caídas de un cliente (se libera su lugar y se avisa) y el cierre del host (`CERRADA` para todos). Al iniciar manda `CUENTA_REGRESIVA` y, a los 3 s, `LARGADA`; desde ese momento rechaza a quien quiera entrar.

Pantallas:

- **Crear partida:** pide el nombre, levanta el servidor y se conecta a `127.0.0.1`. Si el puerto `7777` está ocupado, muestra el error.
- **Unirse:** nombre e IP. Muestra "Conectando..." y, si falla, un mensaje claro (no se pudo conectar, partida llena, versión distinta, nombre repetido...).
- **Lobby en vivo:** los 5 lugares con su color, nombre y estado; cinco cuadros de color para elegir auto (los que tomó otro jugador quedan deshabilitados y muestran su inicial); botón "Estoy listo"; y, solo para el host, "Iniciar carrera" (deshabilitado hasta que haya 2 jugadores y todos estén listos) más sus IPs de la red para dictárselas al resto.
- **Al iniciar:** todos pasan a la carrera con el nombre y el color elegidos. Si se corta la conexión o el host cierra la partida, vuelven al menú con un aviso.

Decisiones

- **Un solo hilo toca el estado de la partida** (el de lógica del servidor). Los hilos lectores le dejan eventos en una `LinkedBlockingQueue` y él los procesa en orden, así no hacen falta `synchronized` sobre los jugadores. La única excepción es el `PING`, que el lector contesta en el momento porque no toca ningún estado.
- **En el cliente, la red nunca toca LibGDX.** El hilo de red solo encola mensajes; cada pantalla los procesa al principio de su `render()`. Para eso se agregó a `PantallaBase` un gancho `actualizar(delta)`.
- **El host es el primer jugador que se une.** Es el que crea la partida y se conecta a su propio servidor de inmediato. Se aclaró en el protocolo.
- **No se activa `SO_REUSEADDR` en el servidor.** En Windows permitiría que otro programa abra el mismo puerto; sin eso, una segunda partida en la misma PC falla con un error claro en lugar de quedar rota.
- **Los mensajes de la red se traducen a un `Mensaje`.** El cliente también genera dos mensajes propios (`_CONEXION_FALLIDA` y `_CONEXION_PERDIDA`, con un guion bajo para no confundirse con los del protocolo), así las pantallas tratan igual lo que llega de la red y las fallas locales.
- **La conexión se mantiene viva también en la carrera y en los resultados.** Si esas pantallas no mandaran `PING`, el servidor daría de baja al jugador a los 6 s. Se verificó (ver abajo).

Problemas encontrados y cómo se resolvieron

1. **La primera corrida de la prueba dio un fallo, que era un error de la prueba y no del servidor.** El servidor había aceptado el cambio de auto y difundido el lobby correcto (se comprobó en el log). La prueba leyó por error un mensaje `LOBBY` viejo que todavía estaba en la cola del cliente, que es una cola en orden de llegada. Se corrigió la prueba para que espere a que se vea el cambio.

2. **"Conectado a la partida de ?"**. Al mirar la captura del lobby de un invitado, el texto con el nombre del host se calculaba antes de que llegara el primer `LOBBY`. Se cambió para que se actualice cada vez que cambia el lobby.
3. **Un mensaje de error largo se salía del borde** de la pantalla en el formulario de unirse; se acortó.
4. **Una decisión que evitó un error futuro**: al iniciar, cada cliente corre una carrera local que dura unos 35 segundos y la sesión seguía abierta; sin `PING` en esas pantallas, el servidor habría dado de baja a todos a los 6 segundos. Por eso `PantallaCarrera` y `Resultados` también atienden la red.

Verificación

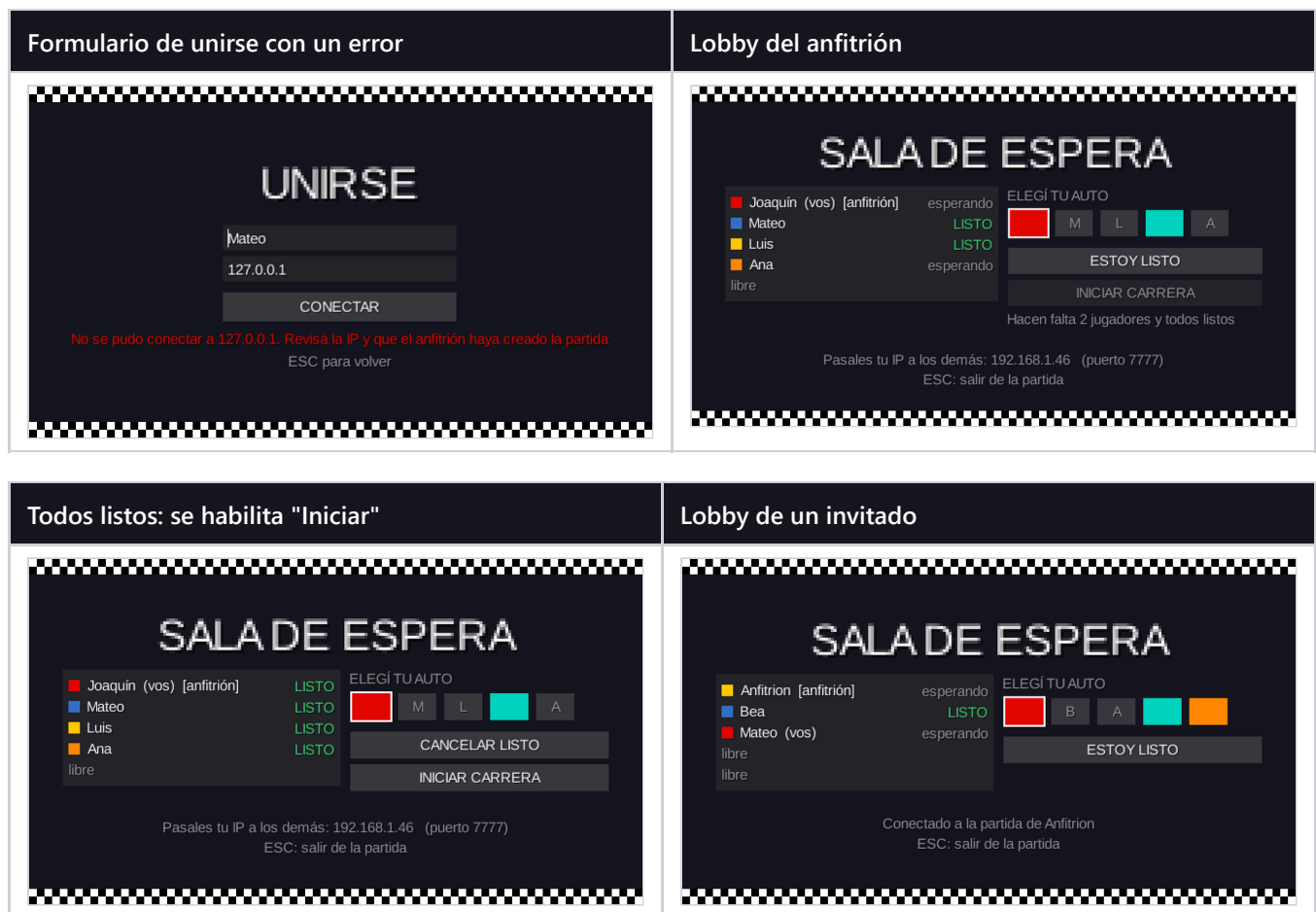
Capa de red, con un servidor real y clientes falsos (una prueba descartable, sin pantalla). 40 comprobaciones, todas correctas:

Grupo	Qué se comprobó
Lobby	El host recibe el id 0 y se lo marca como host; un nombre con tilde (<code>Joaquín</code>) llega bien por UTF-8; el segundo jugador recibe el id 1 y un auto distinto
Errores de <code>UNIRSE</code>	Nombre repetido (sin distinguir mayúsculas), nombre con un símbolo, nombre de 13 caracteres, versión distinta, partida llena (el sexto jugador) y carrera ya empezada, todos con su código de error; el servidor cierra la conexión después de rechazar
Autos	Cada jugador recibe un auto distinto; elegir el auto de otro da <code>AUTO_OCUPADO</code> ; un auto que se liberó se puede elegir
Inicio	<code>INICIAR</code> de alguien que no es host da <code>NO_PERMITIDO</code> ; con jugadores sin marcar da <code>FALTAN_JUGADORES</code> ; con un solo jugador tampoco deja iniciar; con todos listos, todos reciben <code>CUENTA_REGRESIVA</code> , y la <code>LARGADA</code> llega a los 3.009 ms
Durante la carrera	Un jugador que sale genera <code>SALIO</code> para los demás; elegir auto da <code>NO_PERMITIDO</code>
Cierre	Cuando el host sale, todos reciben <code>CERRADA</code> y después ven la conexión cerrada; si el host cierra con <code>SALIR</code> , pasa lo mismo
Caídas	Un cliente que se corta sin avisar libera su lugar en el lobby; un cliente mudo se da de baja a los 6.044 ms (el timeout es 6.000); los que mandan <code>PING</code> siguen conectados después de 9 s
Robustez	Líneas vacías, mensajes desconocidos, campos vacíos, un <code>LOBBY</code> mandado por un cliente y un <code>UNIRSE</code> sin campos: el servidor no se cae y sigue contestando el <code>PING</code>
Fallas de conexión	Un puerto cerrado da <code>CONEXION_FALLIDA</code> en 12 ms; una IP que no responde, en 2.862 ms (el timeout es 3.000); un puerto ocupado lanza <code>BindException</code>
Formato	El ejemplo de mensaje <code>LOBBY</code> del protocolo se lee bien y armar y leer es reversible

Interfaz de punta a punta, con la ventana oculta, usando el flujo real y tres jugadores falsos que se conectan al servidor del host: el menú muestra el aviso; unirse a un puerto vacío muestra el error; un nombre inválido muestra su mensaje; "Crear" lleva al lobby con el host; entran 3 jugadores; el host marca listo y el botón de iniciar se habilita; al iniciar pasa a la carrera; **a los 7,8 s de carrera sigue conectado** (el `PING`

funciona); un jugador sale y el host sigue en la carrera; al volver al menú con ESC, los demás reciben CERRADA ; y un invitado ve el lobby y sale con ESC.

Capturas



Cómo probarlo

Tres instancias en la misma PC (Eclipse, Lwjgl3Launcher , Run As, tres veces):

1. En la primera: "Crear partida", con un nombre.
2. En la segunda y la tercera: "Unirse a una partida", con otro nombre cada una y la IP 127.0.0.1 .
3. En las tres: elegir un auto y "Estoy listo". En la primera, "Iniciar carrera".
4. Con la consola de Eclipse se ve el log de cada mensaje (Config.DEBUG_RED).

Dos PC: la PC del host muestra en el lobby su IP de la red (por ejemplo 192.168.1.46); la otra la escribe en "Unirse". Si no conecta, revisar el firewall de Windows de la PC del host (tiene que permitir Java en redes privadas, o el puerto TCP 7777) y que la red del colegio no aísle a los equipos entre sí. Desde la otra PC se puede comprobar con Test-NetConnection 192.168.1.46 -Port 7777 en PowerShell. La guía recomienda probar cuanto antes en las PC del colegio: si la red bloquea algo, mejor enterarse ahora.

Commits

- Los commits de la etapa están en la rama `feature/lobby-tcp` .